QueryMind AI

Natural Language Database Assistant

Requirements Specification Document

(Business Requirements Document)

Prepared by:
Deep Soni- 202512089
Ishpreet Kaur Bhusari- 202512104
Neel Shah- 202512114
Harsh Rathod- 202512086
Nishit Gal- 202512002
Parshwa Jain- 202512096
Heer Sachdev- 202512117
Bansari Akhani- 202512032

1. Introduction.....3
1.1 Purpose.....3
1.2 Scope of This Revision.....3
1.3 Intended Audience.....3
2. Functional Requirements.....4
3. Non-Functional Requirements.....6
4. User Stories.....7
5. Use Case Diagram.....9
6. Interface Requirements.....9
Chat Interface (Web Client).....9
Admin Panel (Web Client).....9
Backend API (Internal).....9
Database Connectivity Interface.....10
LLM / External Service Interface.....10
7. Wireframes / UI-UX Mockups (Initial Draft).....10
8. Acceptance Criteria.....18
8.1 Query Understanding & SQL Generation.....18
8.2 SQL Validation & Security.....18
8.3 Query Review & Execution.....18
8.4 Multi-Database Connectivity.....19
8.5 Authentication & Role-Based Access.....19
8.6 Visualisation & Export.....19
8.7 Conversational Follow-up.....19
8.8 Admin Dashboard & Query History.....19
9. Future Scope.....19

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements, interface requirements, and acceptance criteria for QueryMind AI, a web-based assistant that lets non-technical users retrieve information from a connected database by asking a question in plain English or Hinglish . It expands the scope defined in the project proposal into requirements that are specific and testable enough to drive design and development.

1.2 Scope of This Revision

This revision covers functional requirements, non-functional requirements, user stories, interface requirements, and acceptance criteria for all in-semester modules described in the project proposal. Use case diagrams and wireframes/UI-UX mockups are intentionally out of scope for this revision; placeholder sections are included below to be completed in a subsequent revision.

1.3 Intended Audience

- The eight project team members, for module-level design and implementation.
- Course faculty/evaluators, for review against the stated deliverable.
- Future contributors extending the system beyond this semester's scope.

2. Functional Requirements

.

ID	Feature	Requirement
FR-01	User Registration and Login	The system shall allow users to register, log in, and securely access the application.
FR-02	Role-Based Access Control	The system shall provide Admin, Analyst, and Viewer roles with different permissions.
FR-03	Database Connection Management	The system shall allow authorized users to connect MySQL, MongoDB, and PostgreSQL databases.
FR-04	Schema Discovery	The system shall automatically retrieve the schema structure, collections/tables, fields/columns, data types, and available relationships from connected databases.
FR-05	Schema Access Control	The Admin shall be able to control which tables and columns are available for querying.
FR-06	Text Query Input	The system shall allow users to enter databThe system shall allow users to export query results in PDF format.ase requests in natural English or Hinglish.
FR-07	Voice Query Input	The system shall allow users to speak queries in English or Hinglish and convert speech into text for processing.
FR-08	Natural Language Query Processing	The system shall identify the user's intent and required database operation from the provided query.
FR-09	Text-to-SQL Generation	The system shall generate the appropriate database query from natural language input using an LLM and the discovered database schema. For SQL-based databases, the generated query shall be in SQL format.
FR-10	SQL Query Review	The system shall display the generated SQL query to the user before execution and allow the user to accept or reject it.
FR-11	SQL Validation and Security	The system shall validate generated queries and block destructive operations, unauthorized tables/columns, and SQL injection attempts.

ID	Feature	Requirement
FR-12	Query Execution	The system shall execute only approved and validated queries with appropriate row and execution limits.
FR-13	Query Result Display	The system shall display query results in a structured tabular format.
FR-14	Result Export	The system shall allow users to export query results in PDF format.
FR-15	Conversational Follow-up	The system shall maintain session-level context to support follow-up queries without requiring users to repeat the complete request.
FR-16	Query History	The system shall maintain user-specific history of natural language queries, generated SQL, and execution details.
FR-17	Admin Dashboard	The system shall provide administrators with database connection, schema mapping, user, role, and query-log management features.
FR-18	Query Logging	The system shall record relevant query activities including user, query, generated SQL, status, and execution information.

3. Non-Functional Requirements

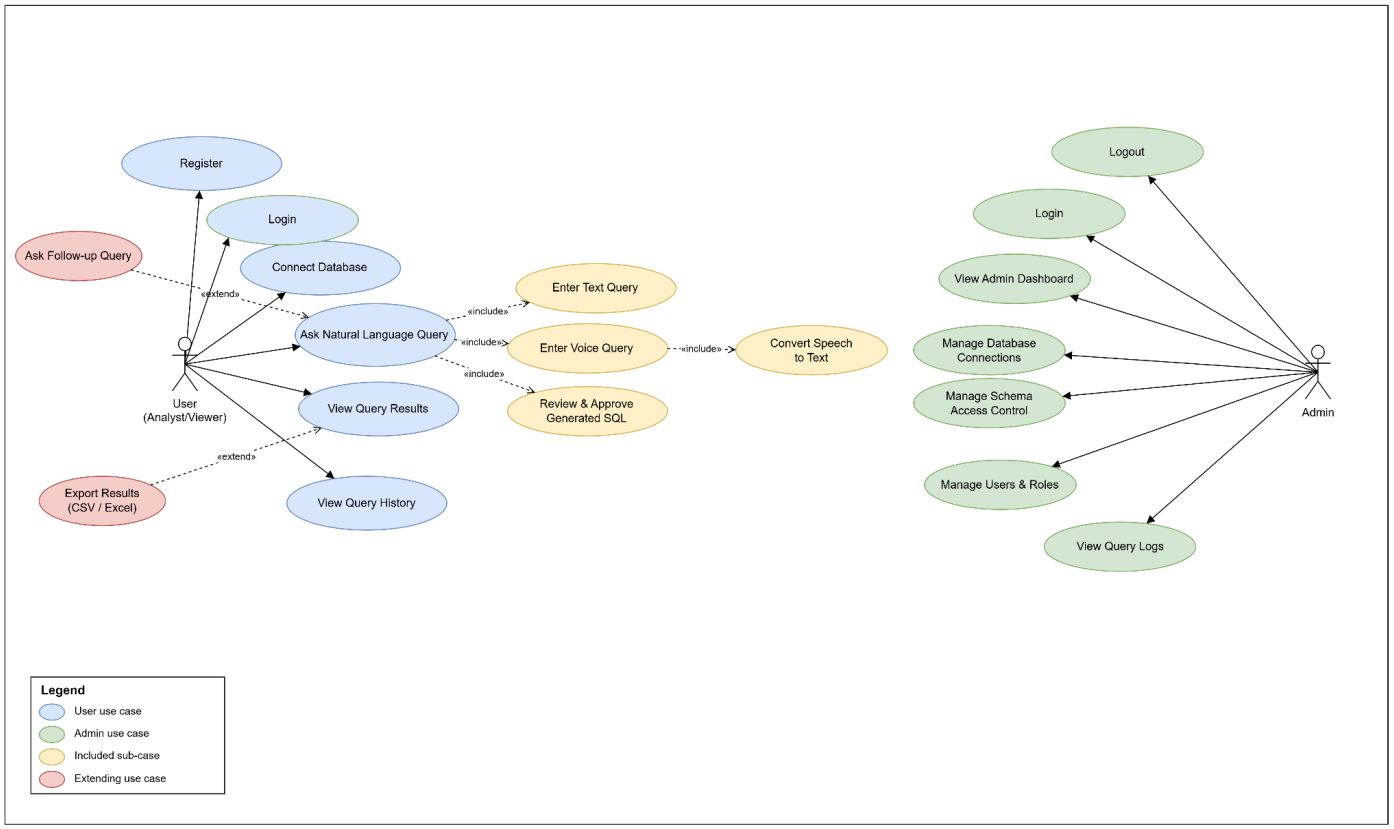
ID	Category	Requirement
NFR-01	**Performance**	The system shall process user queries and generate SQL within a reasonable response time under normal operating conditions.
NFR-02	**Security**	The system shall protect user credentials, database connection information, query data, and other sensitive application information from unauthorized access.
NFR-03	**Reliability**	The system shall handle invalid queries, database errors, LLM errors, and failed connections without crashing the application.
NFR-04	**Scalability**	The system shall be designed to support increasing numbers of users, queries, and connected databases without major architectural changes.
NFR-05	**Usability**	The system shall provide a simple and intuitive interface for users with limited or no SQL knowledge.
NFR-06	**Accuracy**	The system shall generate SQL queries that correctly represent the user's intended natural language request and the connected database schema.
NFR-07	**Availability**	The deployed application shall remain accessible to users during normal operating conditions.
NFR-08	**Compatibility**	The system shall support modern web browsers and work with MySQL, MongoDB, and PostgreSQL database systems
NFR-09	**Maintainability**	The system shall use a modular architecture that allows individual components such as NLP, database connectivity, security, query processing, and result presentation to be modified independently.
NFR-10	**Extensibility**	The system shall be designed so that additional languages, database engines, LLM providers, and result presentation features can be added in the future.
NFR-11	**Data Integrity**	The system shall preserve the accuracy and consistency of stored user information, query history, logs, and generated results.
NFR-12	**Privacy**	The system shall minimize the storage of sensitive database information and protect user and database-related data from unauthorized access.
NFR-13	**Resource Efficiency**	The system shall use reasonable CPU, memory, storage, and network resources to remain suitable for free or low-cost deployment environments.
NFR-14	**Error Handling**	The system shall provide clear and understandable error messages for invalid input, unsupported queries, connection failures, and execution errors.

4. User Stories

ID	User Story	Priority
US-01	As a user, I want to register and log in securely so that I can access the system according to my assigned role.	High
US-02	As an authorized user, I want to connect a MySQL or PostgreSQL database so that I can query its data through the application.	High
US-03	As a user, I want to ask database questions in English or Hinglish using text or voice so that I can interact with the database without knowing SQL.	Critical
US-04	As a user, I want the system to automatically understand my request and generate an SQL query based on the connected database schema so that I do not have to write SQL manually.	Critical
US-05	As a user, I want to review the generated SQL query and accept or reject it before execution so that I remain in control of database operations.	Critical
US-06	As a user, I want the system to validate queries and block unsafe or unauthorized operations so that the connected database remains protected.	Critical
US-07	As a user, I want to view query results in a structured table with a brief summary and export them as PDF so that I can review and use the retrieved information easily.	High
US-08	As a user, I want to ask follow-up questions without repeating my previous query so that I can interact with the database conversationally.	High
US-09	As an Admin, I want to manage database connections, accessible tables and columns, users, and query logs so that I can control and monitor system usage.	High
US-10	As a user, I want to view my previous queries and generated SQL so that I can review or reuse my query history.	Medium

US-11	As a user, I want the system to ask a clarification question when my query is ambiguous so that it doesn't execute a guessed or incorrect query.	Critical
US-12	As an Admin, I want to control which specific tables, collections, fields, and columns are available for querying so that sensitive data is not exposed to unauthorized users.	High
US-13	As a user, I want my spoken query to be accurately converted to text before processing so that voice input works as reliably as typed input.	Medium
US-14	As a user, I want to be clearly notified if the system cannot generate a response in time so that I know to retry instead of waiting indefinitely.	Medium
US-15	As a user, I want large result sets to be limited or paginated so that the interface remains responsive and usable.	Medium
US-16	As an Admin, I want to confirm before performing destructive actions like removing a database connection or revoking a user so that I don't make irreversible changes by accident.	High
US-17	As a user, I want to see a clear, plain-language error message when my query fails or is unsupported so that I understand what went wrong and how to proceed.	Medium
US-18	As an authorized user, I want to switch between connected databases so that I can query the right data source without needing developer assistance.	Medium

5. Use Case Diagram



6. Interface Requirements

Chat Interface (Web Client)

- Provide a single-page chat window accepting text input, with a microphone control for voice input.
- Display the assistant's interpreted intent and the generated database query as a distinct, clearly labelled block.
- Provide explicit Approve / Edit / Cancel controls before a generated query is executed.
- Render results as a responsive data table by default, along with a brief textual summary of the result.
- Provide a PDF export control attached to each result set.

Admin Panel (Web Client)

- Provide screens for: database connections, schema mapping and access management, user and role management, and query log review.
- Reuse the same authentication session as the chat interface; no separate login flow.
- Present destructive actions (e.g. removing a database connection, revoking a user) behind a confirmation step.

Backend API (Internal)

- Expose a REST API (FastAPI) as the single integration point between the web client and the AI/NLP, validation, and database layers.
- Return structured error responses (error code and plain-language message) for blocked queries, timeouts, and authentication failures.

- Version the API surface so that future clients (e.g. a future mobile client) can integrate without breaking the web client.

Database Connectivity Interface

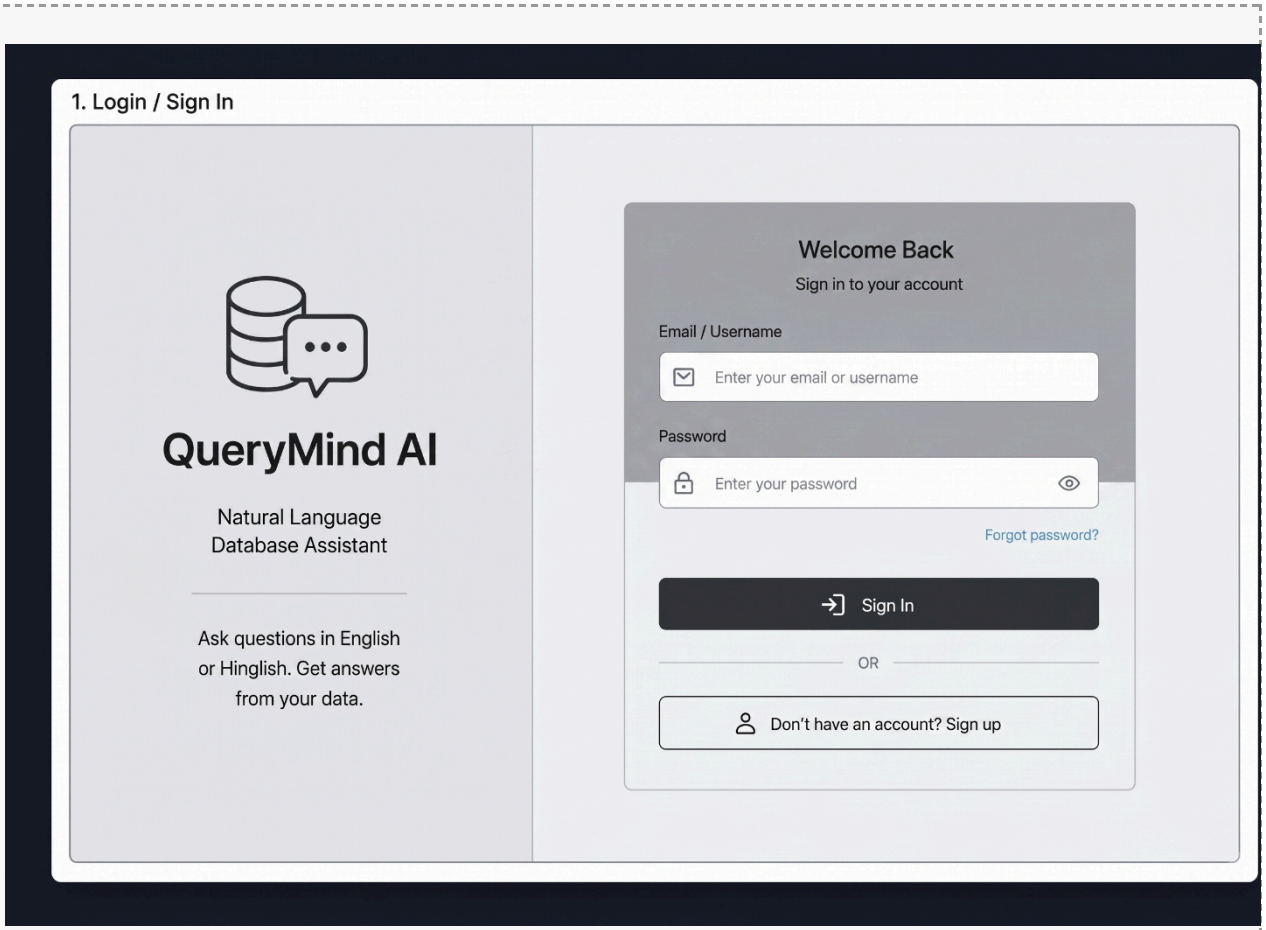
- Provide a database connectivity layer that supports MySQL, MongoDB, and PostgreSQL through appropriate database drivers or libraries..
- Accept database connection details (host, port, credentials, database name) only through the admin panel, never through end-user chat input.

LLM / External Service Interface

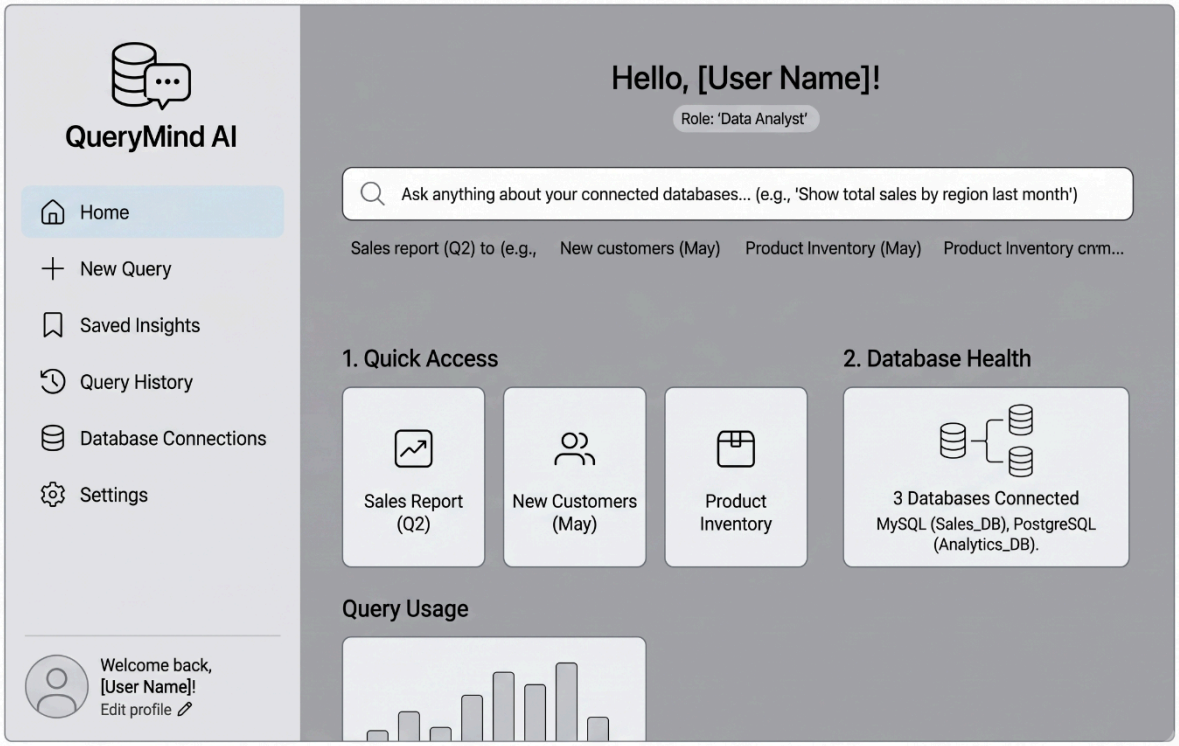
- Integrate with an LLM through an API (GPT, Gemini, or Llama, subject to access and cost) via a swappable provider interface, so the underlying LLM can be changed without affecting other layers.
- Time out and fail safely (no partial or unvalidated SQL reaching the database) if the LLM provider does not respond within a configured limit.

7. Wireframes / UI-UX Mockups (Initial Draft)

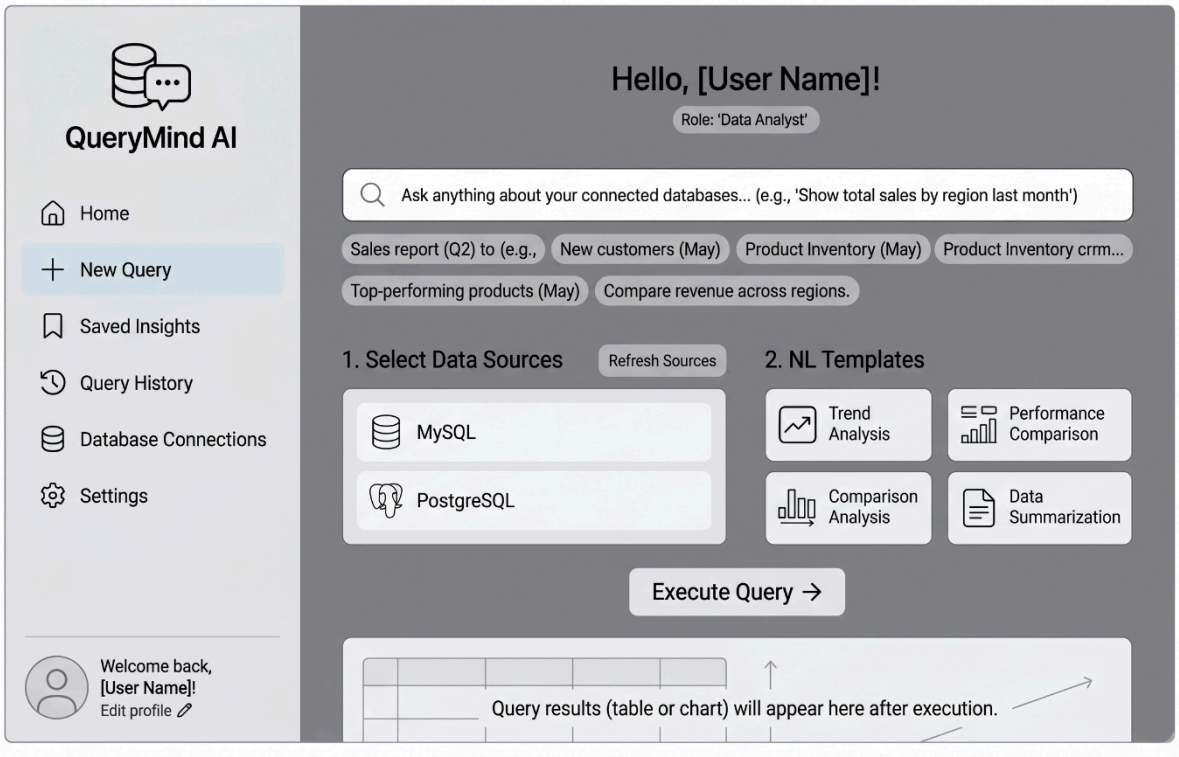
Wireframes for the chat interface, the query review/approval screen, the results view (table and summary), and the admin panel (database connections, schema mapping, user management, and query logs) are to be produced as an initial draft in a subsequent revision.



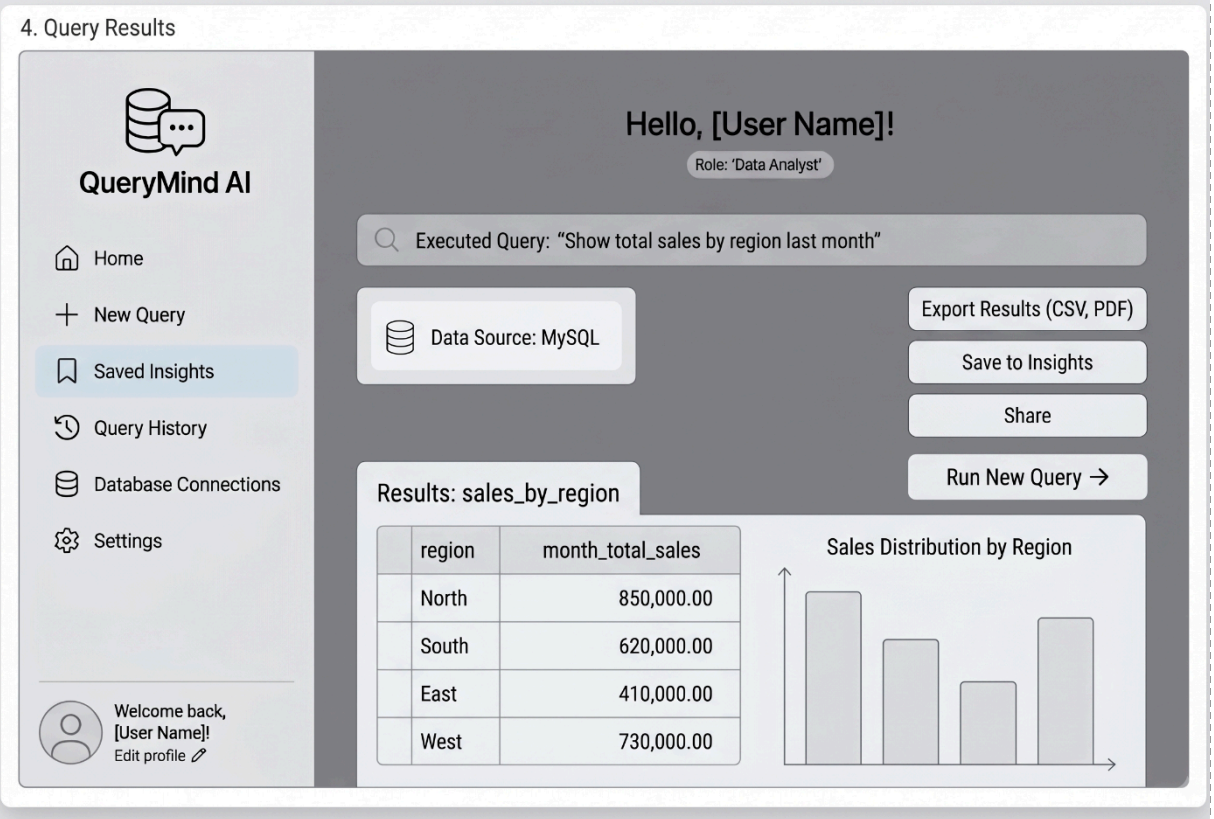
2. Dashboard Overview



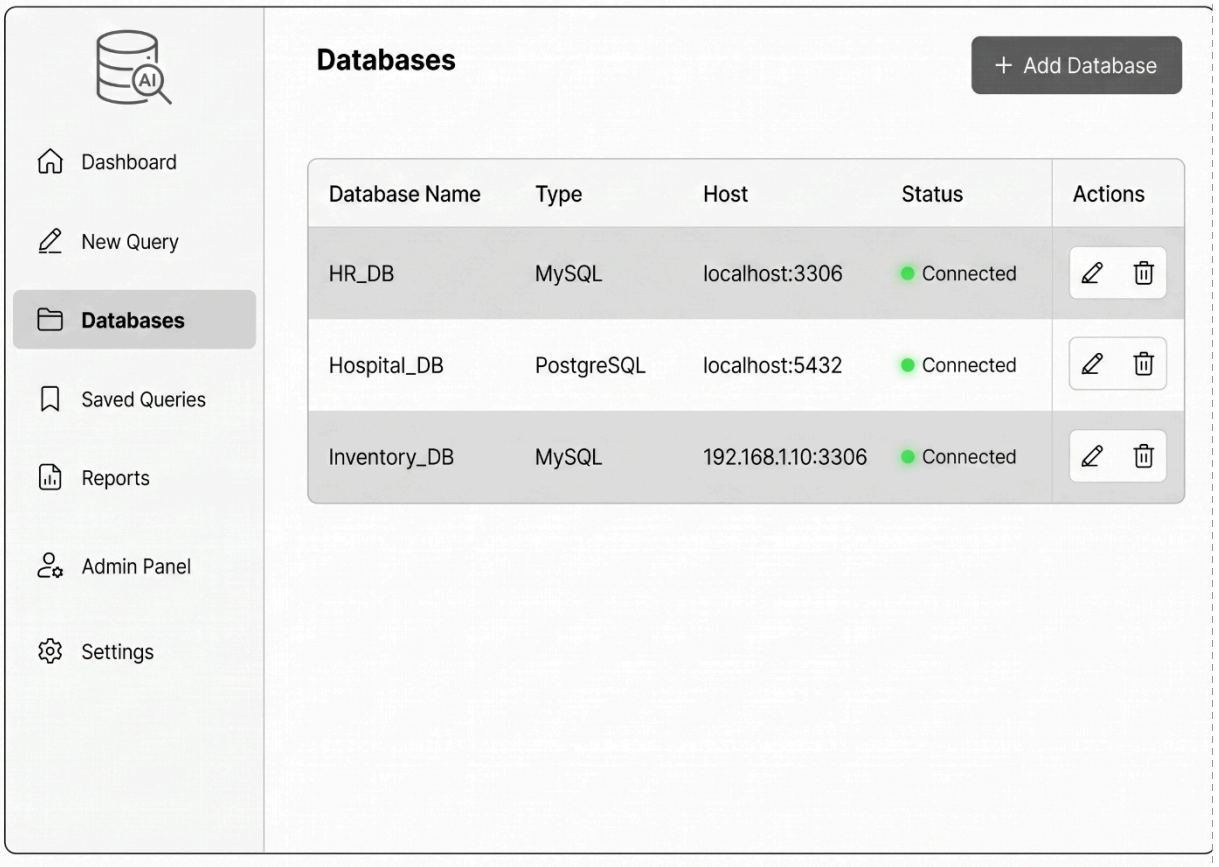
3. New Query



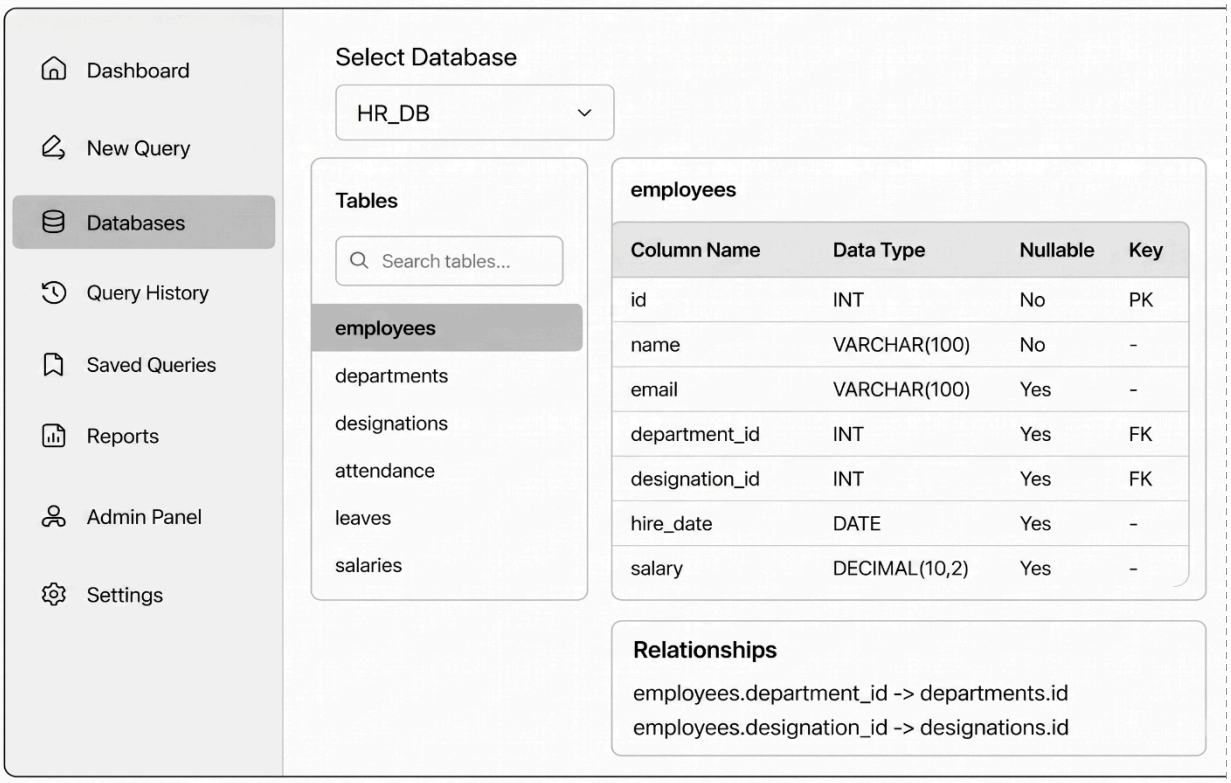
4. Query Results



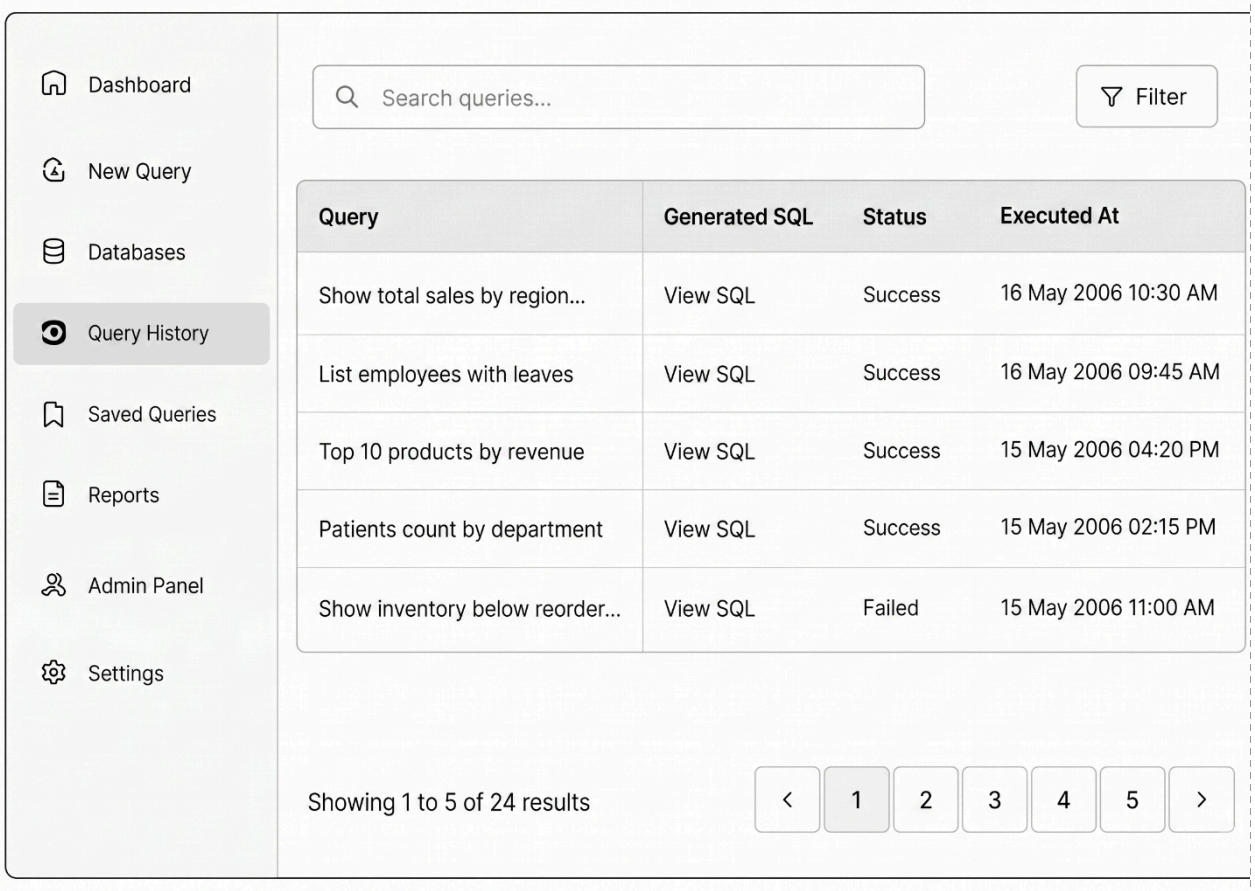
5. Databases



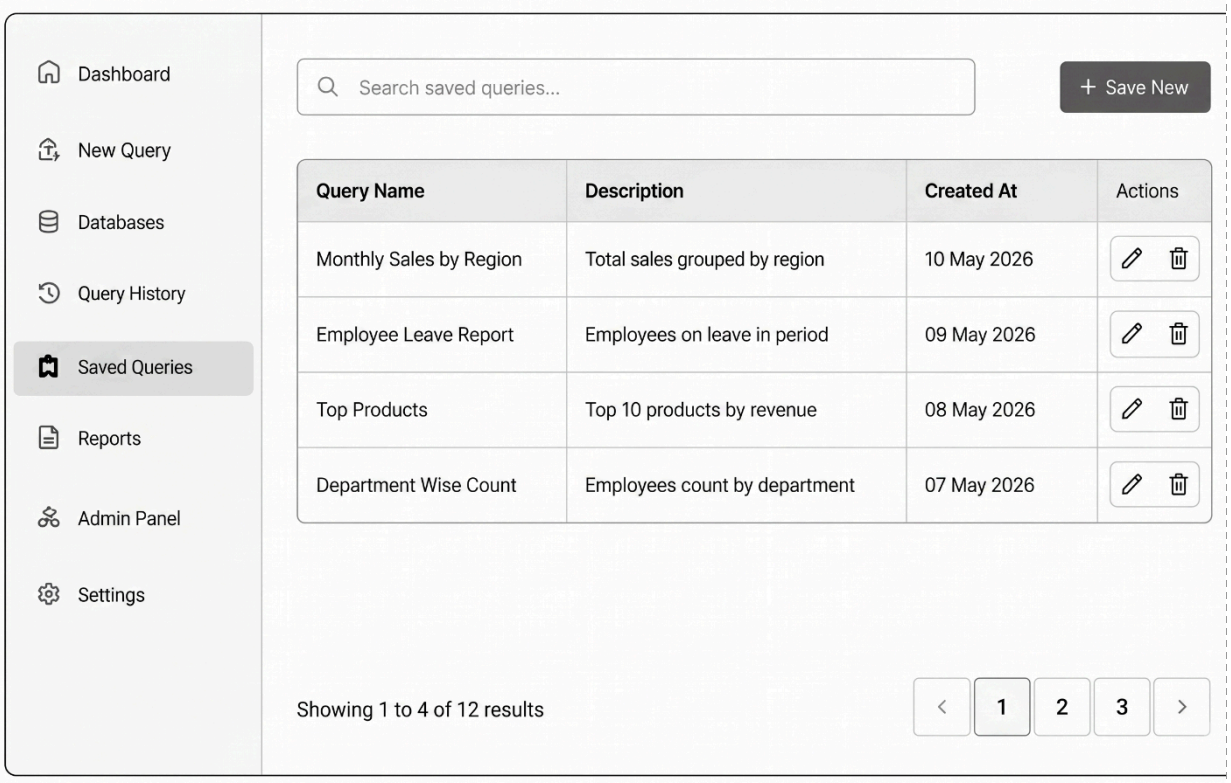
6. Schema Explorer



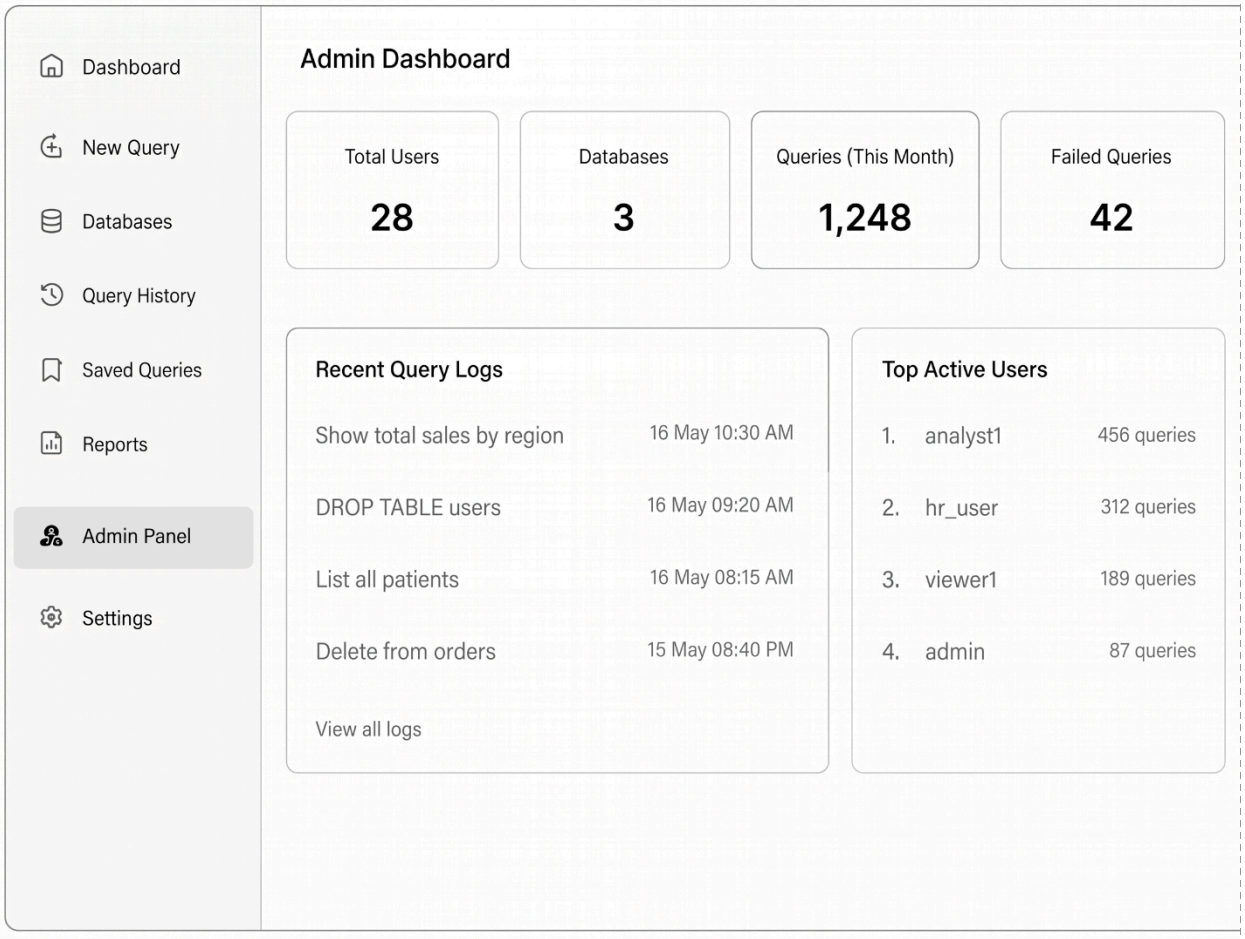
7. Query History



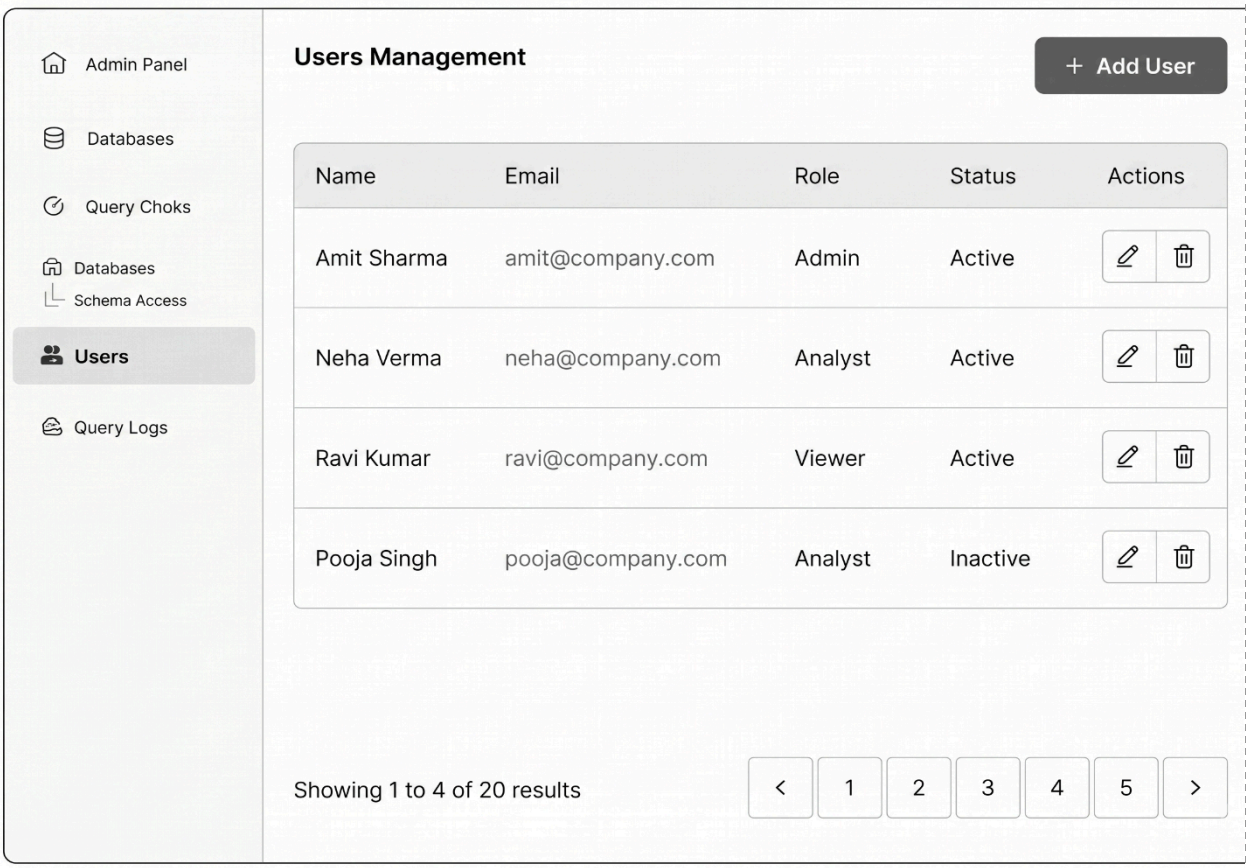
8. Saved Queries

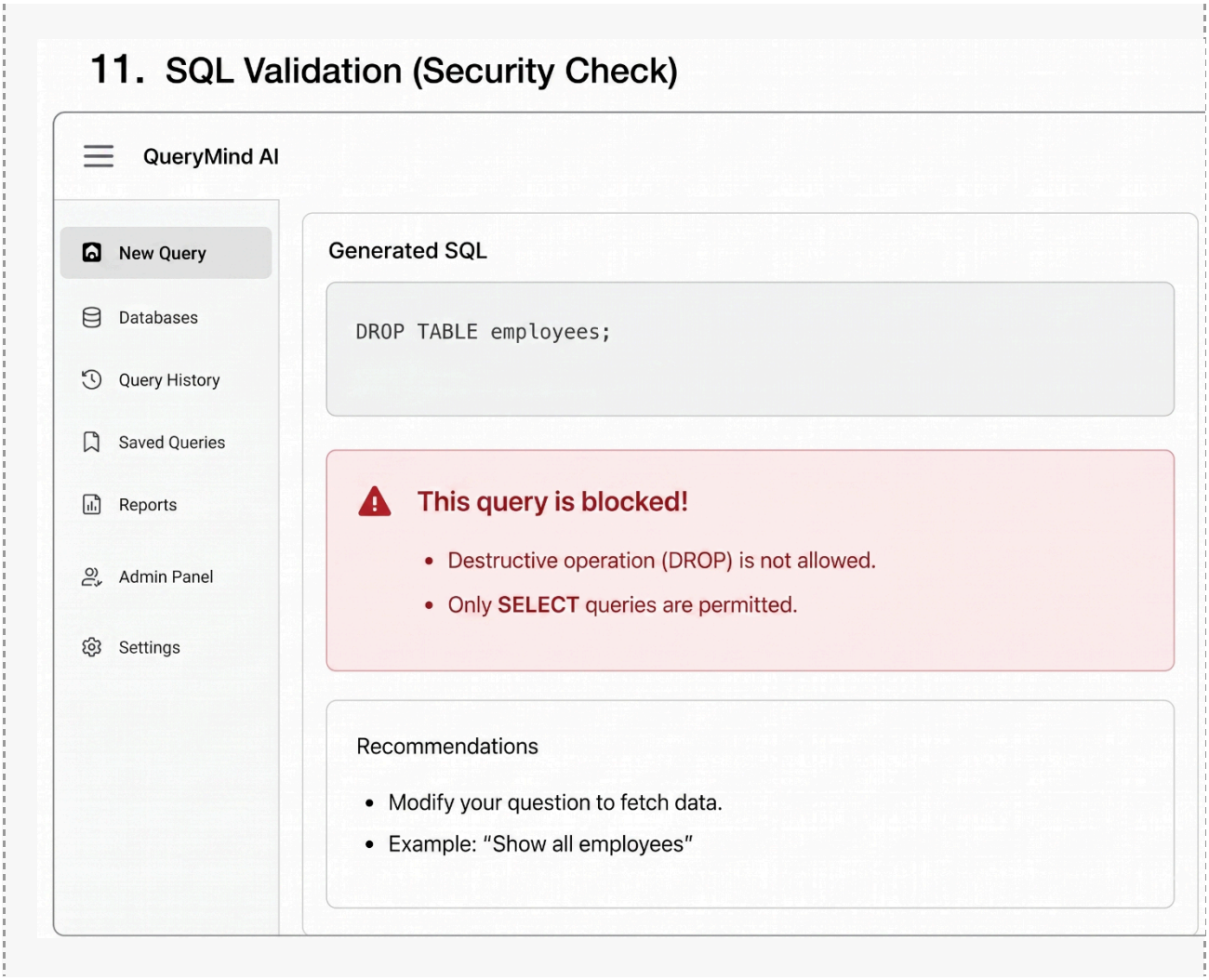


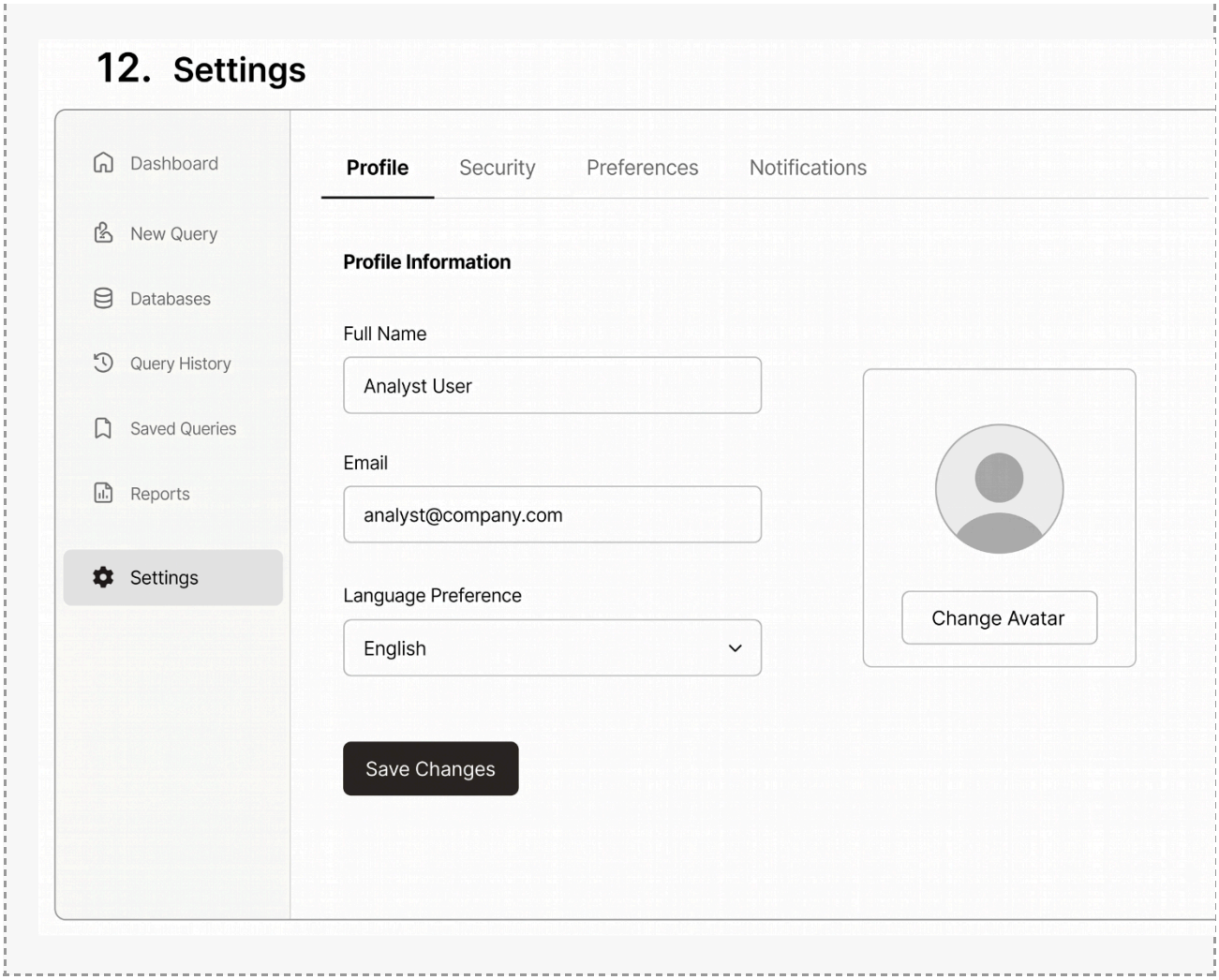
9. Admin Dashboard



10. Users Management




11. SQL Validation (Security Check)
QueryMind AI
New Query
Databases
Query History
Saved Queries
Reports
Admin Panel
Settings
Generated SQL
DROP TABLE employees;
This query is blocked!
Destructive operation (DROP) is not allowed.
Only SELECT queries are permitted.
Recommendations
Modify your question to fetch data.
Example: "Show all employees"



8. Acceptance Criteria

Acceptance criteria are expressed in Given/When/Then form and grouped by feature area. These will be refined into detailed test cases as part of the Testing & Evaluation module.

8.1 Query Understanding & SQL Generation

- Given a natural language query in English or Hinglish, when submitted, then the system displays a generated SQL statement matching the user's intent for at least 80% of the queries in the 150-pair evaluation set.
- Given an ambiguous query, when submitted, then the system asks a clarification question instead of executing a guessed query.

8.2 SQL Validation & Security

- Given a generated or user-edited database query containing a destructive or potentially unsafe operation, when submitted for execution, then the system shall block or require appropriate confirmation according to the configured security rules and show a plain-language reason.
- Given a query attempting a known SQL injection pattern, when validated, then the system rejects it and logs the attempt.
- Given a query requesting more rows than the configured limit, when executed, then the system truncates or paginates the result rather than returning an unbounded set.

8.3 Query Review & Execution

- Given a generated SQL statement, when displayed to the user, then execution does not begin until the user explicitly approves it.
- Given an approved query, when executed against the connected database, then the result is returned to the client within the configured timeout or the user is notified of the timeout.

8.4 Multi-Database Connectivity

- Given a MySQL, MongoDB, or PostgreSQL database configured by an administrator, when an end user selects it as the active data source, then queries shall execute correctly against the selected database type without requiring changes to the user interface.

8.5 Authentication & Role-Based Access

- Given a user with the Viewer role, when they attempt an action reserved for Admin (e.g. connecting a database), then the system denies the action.
- Given valid credentials, when a user logs in, then a JWT is issued and required on all subsequent API requests.

8.6 Result Presentation & Export

- Given an aggregate result set, when the user requests a chart view, then the system renders it as a bar, line, or pie chart as appropriate to the data shape.
- Given any result set, when the user selects export, then a correctly formatted CSV or Excel file downloads containing the full (non-truncated within limits) result.

8.7 Conversational Follow-up

- Given a prior query and result in the same session, when the user submits a follow-up query that references it (e.g. "only for last month"), then the system resolves the follow-up using that context.

8.8 Admin Dashboard & Query History

- Given a completed query execution, when an administrator opens the query log, then the natural language query, generated database query, requesting user, and execution status are visible.
- Given a user's own past queries, when they open query history, then they can view and re-run a prior query.

9. Future Scope

The following are explicitly excluded from this semester's requirements and deliverables, per the project proposal's Future Scope:

- Support for additional regional and international languages beyond English and Hinglish.
- Support for additional database engines beyond MySQL, MongoDB, and PostgreSQL (e.g. Oracle, SQL Server, SQLite).
- Persistent AI chat memory across multiple sessions (only within-session follow-up is in scope).
- Integration with BI tools such as Power BI and Tableau.
- AI-generated database insights and recommendations.
- Real-time conversational voice assistant with speech responses (voice input only, not spoken responses).
- Automated report scheduling and email sharing of query results.